



МИНОБРНАУКИ РОССИИ

**федеральное государственное бюджетное образовательное учреждение
высшего образования**

**«Московский государственный технологический университет «СТАНКИН»
(ФГБОУ ВО «МГТУ «СТАНКИН»)**

**Основная образовательная программа 09.03.02
«Информационные системы и технологии»**

**Отчет по дисциплине «Алгоритмы и структуры обработки данных»
по лабораторной работе №2**

Тема: «Связные списки»

**Проверил
к.т.н. доцент**

Ковалев И.А.

подпись

**Выполнила
студентка группы ИДБ-25-06**

Якунина В.С.

подпись

Оглавление

ВВЕДЕНИЕ.....	3
ГЛАВА 1 ВСПОМОГАТЕЛЬНЫЕ ФУНКЦИИ И КЛАССЫ.....	4
1.1 Базовый класс узла связного списка.....	4
1.2 Функция создания связного списка из массива.....	4
1.3 Функция вывода связного списка в консоль.....	5
1.4 Функция для перевода связного списка в массив.....	5
1.5 Класс односвязного списка с методом добавления нового элемента в конец связного списка.....	6
ГЛАВА 2 ВЫПОЛНЕНИЕ ЗАДАЧ САМОСТОЯТЕЛЬНОЙ РАБОТЫ.....	7
2.1 Задача А6. Функция перемещения первого элемента списка в конец.....	7
2.2 Задача В4. Функция подсчета элементов связного списка, больших среднего арифметического всех элементов.....	8
2.3 Задача С3. Функция группировки элементов связного списка по четным и нечетным значениям.....	9
2.4 Задача D2. Функция поиска узла пересечения и слияния в один двух связных списков.....	10
2.5 Задача Е5. Класс узла полинома.....	11
2.6 Задача Е5. Функция суммы 2 полиномов.....	12
2.7 Задача Е5. Функция вывода полинома в консоль.....	13
ЗАКЛЮЧЕНИЕ.....	14

ВВЕДЕНИЕ

Целью работы является изучение связанных списков, а также обучение их использованию в задачах. В данной работе использовался язык программирования Python. Ссылка на [github](#).

ГЛАВА 1 ВСПОМОГАТЕЛЬНЫЕ ФУНКЦИИ И КЛАССЫ

1.1 Базовый класс узла связного списка

```
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next
```

Данный класс позволяет создать узел связного списка для использования в дальнейшем.

1.2 Функция создания связного списка из массива

```
def build_list(values):
    """Создать список из массива:
    build_list([1,2,3]) ? 1?2?3?None"""
    dummy = ListNode(0)
    cur = dummy
    for v in values:
        cur.next = ListNode(v)
        cur = cur.next
    return dummy.next
```

Данная функция преобразует массив в связный список.

1.3 Функция вывода связанного списка в консоль

```
def print_list(head):  
    """Напечатать список"""  
    parts = []  
    while head:  
        parts.append(str(head.val))  
        head = head.next  
    print(" → ".join(parts) + " → None")
```

Функция выводит связный список в консоль.

1.4 Функция для перевода связанного списка в массив

```
def list_to_array(head):  
    """Список → массив (для проверки)"""  
    result = []  
    while head:  
        result.append(head.val)  
        head = head.next  
    return result
```

Функция переводит связный список в массив для проверки.

1.5 Класс односвязного списка с методом добавления нового элемента в конец связного списка

```
class ListLinked:
    def __init__(self):
        self.head = None

    # добавление элементов в конец списка
    def push_back(self, val):
        new_node = ListNode(val)
        if not self.head:
            self.head = new_node
            return
        cur = self.head
        while cur.next:
            cur = cur.next
        cur.next = new_node
```

Данный класс создает связный список. Далее возможно добавление нового элемента в конец этого связного списка с помощью метода.

ГЛАВА 2 ВЫПОЛНЕНИЕ ЗАДАЧ САМОСТОЯТЕЛЬНОЙ РАБОТЫ

2.1 Задача А6. Функция перемещения первого элемента списка в конец

```
def move_first_to_end(head):  
    '''Переместить первый элемент списка в конец'''  
  
    if not head or not head.next:  
        return head  
  
    first_node = head  
    new_head = head.next  
  
    tail = head  
    while tail.next:  
        tail = tail.next  
  
    tail.next = first_node  
    first_node.next = None  
  
    return new_head
```

Функция перемещает первый элемент списка в самый его конец.

2.2 **Задача В4. Функция подсчета элементов связного списка, больших среднего арифметического всех элементов**

```
def count_above_average(head) -> int:
    '''Подсчитать, сколько элементов списка строго больше
        среднего арифметического всех элементов'''

    if not head:
        return 0

    len_head = 0
    sum_head = 0
    count_el = 0
    arith_mean = 0

    cur = head
    while cur.next:
        len_head += 1
        sum_head += cur.val
        cur = cur.next
    len_head += 1
    sum_head += cur.val
    arith_mean = sum_head // len_head

    cur = head
    while cur.next:
        if cur.val > arith_mean:
            count_el += 1
        cur = cur.next

    if cur.val > arith_mean:
        count_el += 1

    return count_el
```

Функция считает и количество элементов, которые больше среднего арифметического всех элементов связного списка.

2.3 Задача С3. Функция группировки элементов связного списка по четным и нечетным значениям

```
def split_even_odd(head):  
    '''Разделить список на два: в первом – узлы с чётными  
        значениями,  
        во втором – с нечётными. Вернуть оба'''  
  
    if not head:  
        return False  
  
    evens = ListLinked()  
    odds = ListLinked()  
  
    cur = head  
    while cur.next:  
        if cur.val % 2 == 0:  
            evens.push_back(cur.val)  
        else:  
            odds.push_back(cur.val)  
        cur = cur.next  
  
    if cur.val % 2 == 0:  
        evens.push_back(cur.val)  
    else:  
        odds.push_back(cur.val)  
  
    return evens, odds
```

Функция делит исходный связный список на еще 2 связанных списка. В один из них функция добавляет узлы с четными значениями, в другой — узлы с нечетными значениями.

2.4 Задача D2. Функция поиска узла пересечения и слияния в один двух связанных списков

```
def get_intersection(headA, headB) -> int | None:
    '''Два списка могут сливаться в один (Y-образная
        форма).
        Найти узел, где они пересекаются, или `None`'''

    if not headA or not headB:
        return None

    curA = headA
    lenA = 0
    curB = headB
    lenB = 0

    while curA.next:
        lenA += 1
        curA = curA.next

    while curB.next:
        lenB += 1
        curB = curB.next

    curA = headA
    curB = headB
    while lenA > lenB:
        lenA -= 1
        curA = curA.next

    while lenB > lenA:
        lenB -= 1
        curB = curB.next

    while curA.next and curB.next:
        if curA == curB:
            return curA
        curA = curA.next
        curB = curB.next

    return None
```

Функция ищет узел, где начинают сливаться 2 связанных списка.

2.5 Задача E5. Класс узла полинома.

```
class PolyNode:
    def __init__(self, coeff, exp, next=None):
        self.coeff = coeff
        self.exp = exp
        self.next = next
```

Данный класс описывает узел связного списка, в котором хранятся элементы полинома.

2.6 Задача E5. Функция суммы 2 полиномов

```
def add_polynomials(p1, p2):  
    '''  
    Полином представлен списком `(коэффициент, степень)`  
    по убыванию степеней.  
    Сложить два полинома.  
    '''  
  
    if not p1 or not p2:  
        return False  
  
    curp1 = p1  
    curp2 = p2  
    max_exp = max(curp1.exp, curp2.exp)  
  
    pres = PolyNode(0, max_exp)  
    head = pres  
    while max_exp > 0:  
        max_exp -= 1  
        new_node = PolyNode(0, max_exp)  
        pres.next = new_node  
        pres = pres.next  
  
    curpres = head  
    while curpres.next:  
        if curpres.exp == curp1.exp:  
            curpres.coeff += curp1.coeff  
            curp1 = curp1.next  
        if curpres.exp == curp2.exp:  
            curpres.coeff += curp2.coeff  
            curp2 = curp2.next  
        curpres = curpres.next  
  
    if curp1 and curpres.exp == curp1.exp:  
        curpres.coeff += curp1.coeff  
    if curp2 and curpres.exp == curp2.exp:  
        curpres.coeff += curp2.coeff  
  
    return head
```

Функция два полинома.

2.7 Задача E5. Функция вывода полинома в консоль

```
def print_poly(head) -> None:
    cur = head
    while cur.next:
        if cur.coeff != 0:
            print(f'{cur.coeff}x^{cur.exp}', end = ' + ')
        cur = cur.next
    print(f'{cur.coeff}x^{cur.exp}')
```

Функция выводит полином в консоль.

ЗАКЛЮЧЕНИЕ

В данной лабораторной работе мы изучили связные списки и сразу применили их на практике.